notes.arpanadhikari7.com.np

By: Manish Awale

What is NLP



· Understanding human language

· Not only understand single word individually but able to understand the context of those words

· Interactions between computers and humans using natural languages via text, audio, visuals

By: Manish Awale

notes.arpanadhikari7.com.np

Common NLP tasks

Email spam detection

sentences

sentence

• named entity recognition

Classifying each word in a

Generating text content

Classifying whole

• Sentiment analysis •

• Grammatical components identification

• Sentence completion • text generation

Extracting an answer from a text

• Question answering based on context

Generating a new sentence from an input text

• Summarization • language translation

• NLP isn’t limited to written text only.

• It also tackles complex challenges in speech recognition and computer vision •

E.g. generating a transcript of an audio sample or a description of an image.

By: Manish Awale

notes.arpanadhikari7.com.np

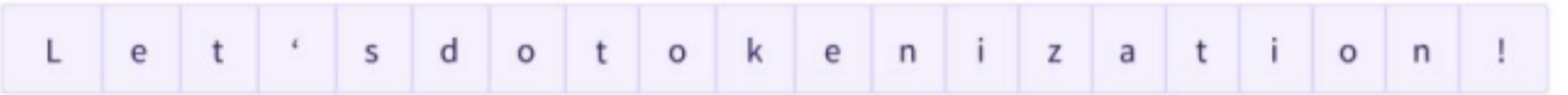
Tokenization

Segmentation of text into smaller units called tokens

Token: The atomic unit of text

Types of Tokenization

Character Tokenization



· Each character is a token

[redacted]

· Very small vocabulary size including all possible characters

· No issue for out-of-vocabulary

[redacted]:

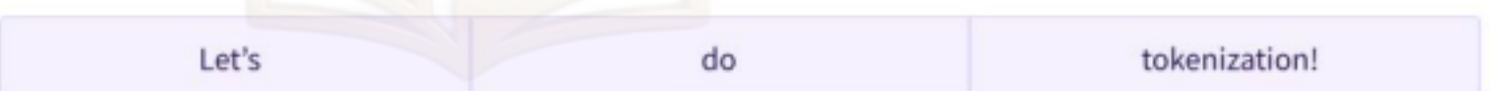
· Much larger input length significantly slowing inference time.

· Learned token embedding are not meaningful.

By: Manish Awale

Word Tokenization

· Each word is treated as a token



notes.arpanadhikari7.com.np

[redacted]

· Easy to understand and interpret

· Test decomposed into fewer tokens which speeds up model inference time.

[redacted]:

· Large vocabulary size, hurting efficiency and Out-of-vocabulary words are not handled.

Subword Tokenization

Each word is divided into sub units that are treated as tokens. e.g. BPE, WordPiece.

[redacted]

· Reduced vocabulary size which captures meaningful word roots.

· Handles out-of-vocabulary words better.

[redacted]:

· Implementation is more complex.

· Increased input length slowing down inference.



By: Manish Awale

notes.arpanadhikari7.com.np

Vocabulary

· A vocabulary V is a fixed set of predefined tokens.

· It is used as a reference to convert the input text into tokens.

· Vocabularies associate each distinct token value with a unique index.

· Each token is represented as a vector in a high-dimensional space

Special tokens

· Conveys a specific meaning

[PAD] : Padding
Hello there ! [PAD] ... [PAD]

· Fills the rest of the sentence until the maximum sequence length n_{max} · Useful to batch computations to speed up

computations

[UNK] : Unknown

I am [UNK] .

· Represents an entity in the input text that was not part of the vocabulary.

By: Manish Awale

Tokenizer

notes.arpanadhikari7.com.np

· Tokenizer, T is a process that converts an input text into tokens so that they can later be represented by vectors for a model to be able to process it.

· Text ↔ Tokens

· Encode: converts text into tokens · Decode: converts tokens back to text format

Normalization

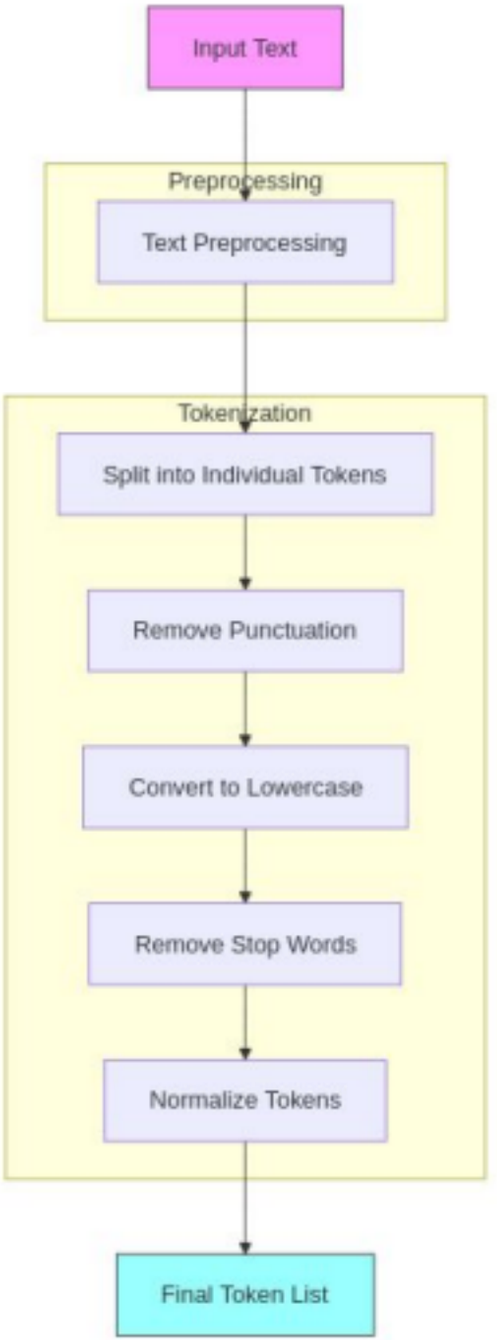
Available on notes.arpanadhikari7.com.np

he is reading.

he is reading. --> *T* --> he is reading . he is reading . --> *T* -->

· To deal with inconsistencies in the input text such as variation in spelling, punctuation, capitalization

notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Breaking text into words/tokens

Removing common words that don't add meaning (like "the," "a," "is," "and")

notes.arpanadhikari7.com.np

[redacted]: chop off word endings

running → run, fishing → fish, argued → argu, studies → studi

[redacted]: converts to dictionary base forms

running → run, fishing → fish, argued → argue, studies → study, better → good

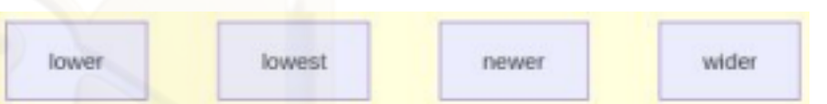
By: Manish Awale

Subword Algorithms

BPE (Byte-Pair Encoding): A tokenization method that constructs the vocabulary by learning from the most common pair of entities that are present in the training corpus.

Tokenizer Training

First, a tokenizer is trained on a given corpus.

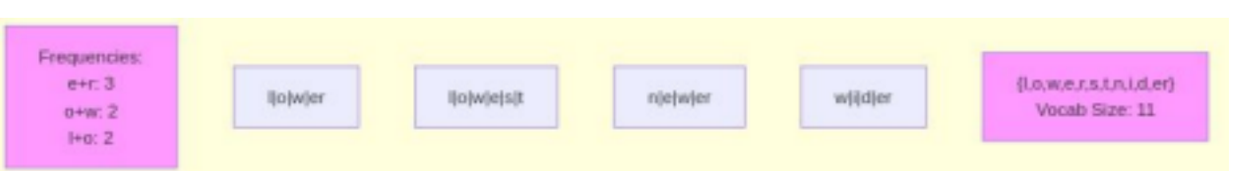


- [redacted]
  - Divide the corpus into characters and count the occurrence of each character.
  - The initial vocabulary is composed of the unique characters that appeared in the corpus and has a size v_i



notes.arpanadhikari7.com.np

- [redacted]
  - First, count the frequency of token pairs in the initial corpus
  - Select pair that occurred most
  - Then merge the pair and add to vocabulary.





Tokenizer Training

- [redacted]
  - Add new elements until the vocabulary reaches the desired size of $|V| = v_f > v_i$
    - Also, add any predefined tokens that handle special cases e.g. [UNK] {l,o,w,e,r,s,t,n,i,d,er,ow,[UNK]}



Encoding

Trained tokenizer is used to convert text to tokens.

- Divide into characters
- Apply merge rules: Start by applying the first merge rules in the order of insertion in the vocabulary. Then successively apply the remaining merge rules.

notes.arpanadhikari7.com.np

Applications: **GPT, LLaMA** (Huggingface BPE tokenization example)

Variations: **WordPiece (**used in BERT**) -** merges based on the most likely pair

By: Manish Awale

Token Embeddings

Vector representation for each token of the vocabulary

· ***One-hot encodings***

A method that represents each token, *i* of vocabulary with a vector of length *N* where the ith element is 1 and all other elements 0. Pros:

token i → *i* → 0 … 0 1 0 0 … 0

- Easy to construct

- Can be used directly by neural networks

Cons:

- Token similarity: similarity between any two tokens are orthogonal, which means it cannot accurately express the similarity between different words. Similar tokens should be closer and dissimilar tokens should be farther.
- Dimensionality: dimension of each token would be as large as vocabulary size.

By: Manish Awale

Available on notes.arpanadhikari7.com.np

Token Embedding

The purpose of embeddings is not only to get the vector representation but to understand the meaning of a token and their relationship with other tokens in the vocabulary.

notes.arpanadhikari7.com.np

Embedding visualization [https://projector.tensorflow.org/]

By: Manish Awale

- ***Word2vec***

Word2vec is a family of methods that aims at generating token embeddings having, - Fixed dimension: dimension d_{model} of the embedding vector is fixed irrespective of the size of the vocabulary. (generally 100 to 300)

d_{model}

0.5 -5.4 0.2 1.3 2.0 0.9 -3.8 1.3

token i

Input Layer:

- One hot encoded vector (vocabulary size) **Hidden Layer:**

- Dense layer (embedding dimension) **Output Layer:**

- Softmax layer (vocabulary size)

[redacted] $|V|$ [redacted] $_{model}$

notes.arpanadhikari7.com.np

[redacted] d_{model} [redacted]

x_{OHE} Projection x_{embed} Prediction $\hat{y}$ $\mathcal{L}$

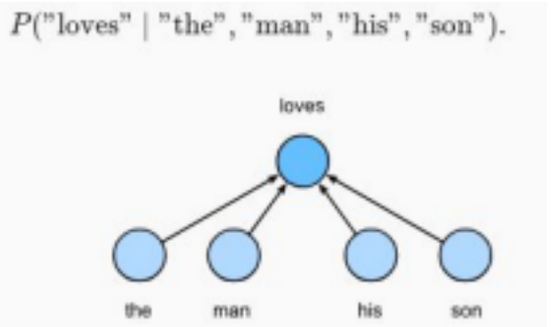
y_{OHE}

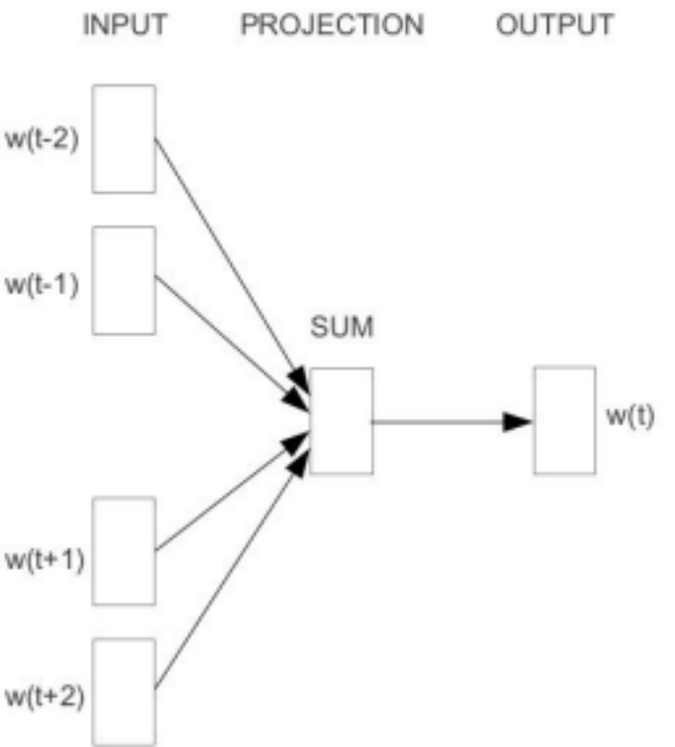
By: Manish Awale

- ***CBOW (Continuous Bag of Words)***

- CBOW is a word2vec model that predicts a center token based on its surrounding context tokens. - The idea is to train a model that predicts a center token given its surrounding context tokens and once the model is trained, learned embeddings are represented as tokens.

notes.arpanadhikari7.com.np





Training

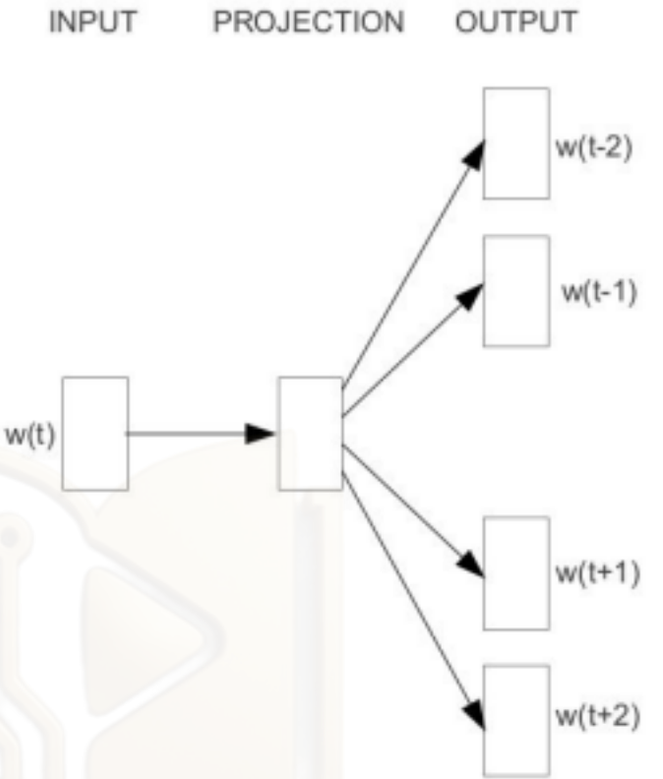
- Initialize network: Initialize a network with one hidden layer with two weight matrices, W1: input -> hidden, W2: hidden -> output - Input Processing: Take multiple context words and get average OHEs - Forward pass: Predict the target token

By: Manish Awale

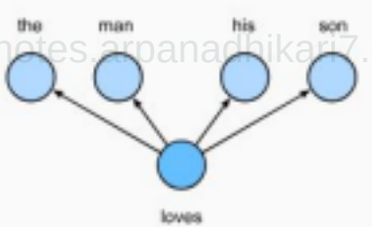
○ *Skip-gram*

- Skip-gram is a word2vec model that predicts a context tokens based on center token. - The idea is to train a model that predicts context tokens given the center tokens and once the model is trained, learned embeddings are represented as tokens.

notes.arpanadhikari7.com.np



$$P(\text{"the"} \mid \text{"loves"}) \cdot P(\text{"man"} \mid \text{"loves"}) \cdot P(\text{"his"} \mid \text{"loves"}) \cdot P(\text{"son"} \mid \text{"loves"}).$$



notes.arpanadhikari7.com.np

Training

- Initialize network: Initialize a network with one hidden layer with two

weight matrices, W1: input -> hidden, W2: hidden -> output

- Input Processing: For each word-pair, take center word and get OHE
- Forward pass: Predict each of the context tokens

By: Manish Awale

○ ***Skip-gram***

- basic example [redacted]

By: Manish Awale

Document Embedding

- Vector representation for an entire text preserving semantic relationship
- Embeddings can be used in tasks like document similarity, document classification, information retrieval

○ ***BOW (Bag of Words)***

BOW takes into account the frequency at which each token appears and relies on their associated word OHE representation

notes.arpanadhikari7.com.np

- All tokens in the document are included in the vocabulary V.

- Fore each word, the number of times it appeared in the document is associated - Each document is represented by a vector of dimension |V|, which is the sum of word frequencies associated with their word embeddings.

By: Manish Awale

Original Documents:

1. The cat sat on the mat
2. The dog sat on the log
3. The cat and dog are friends
4. Machine learning is fascinating
5. Deep learning uses neural networks

Vocabulary (18 words):

BOW Example

notes.arpanadhikari7.com.np

['and', 'are', 'cat', 'deep', 'dog', 'fascinating', 'friends', 'is', 'learning', 'log', 'machine', 'mat', 'networks', 'neural', 'on', 'sat', 'the', 'uses']

Document 1: 'The cat sat on the mat'

Embedding vector shape: (18,)

Embedding vector: [0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 2 0]

Non-zero elements: 5

Vector sum (total word count): 6

Document 4: 'Machine learning is fascinating...'

Embedding vector shape: (18,)
Embedding vector: [0 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 0 0]
Non-zero elements: 4
Vector sum (total word count): 4

Document 2: 'The dog sat on the log'

Embedding vector shape: (18,)
Embedding vector: [0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 1 2 0]
Non-zero elements: 5
Vector sum (total word count): 6

Comparing Document 1 and Document 2:
cosine_similarity: 0.7500

Comparing Document 1 and Document 4:
cosine_similarity: 0.0000

By: Manish Awale

BOW (Bag of Words)

○ *Advantages*

- Each dimension maps to a word, making vectors interpretable
- Captures word frequency, not just presence
- Can be used with clustering, similarity, and classification

notes.arpanadhikari7.com.np

○ *Limitations*

- No semantic understanding as it treats words as independent and ignores the order of words - Sparse representations with many zero elements results in computational inefficiency with large datasets.

By: Manish Awale

○ *TF-IDF (Term Frequency - Inverse Document Frequency)*

TF-IDF takes into account for both the frequency of a word in a given document and its prevalence across all documents.

- Term frequency:

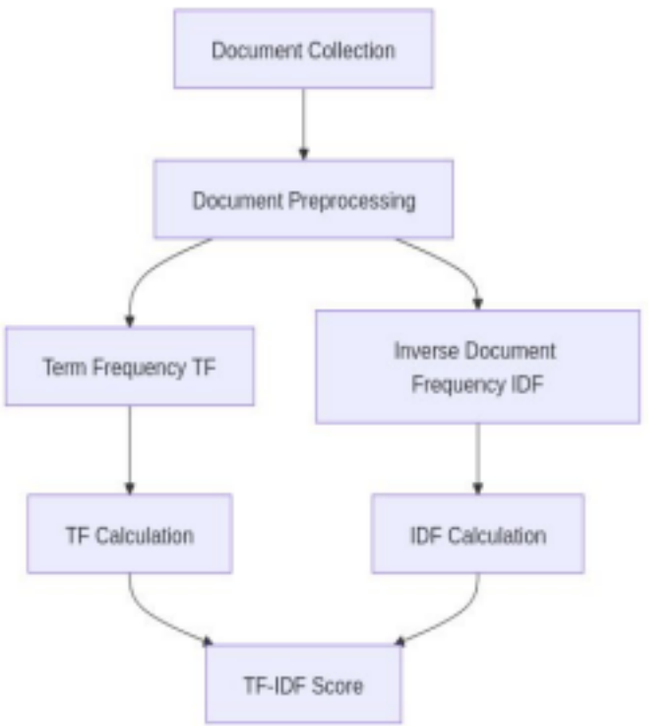
$$TF(t,d) = \frac{\text{(Number of times term t appears in document d)}}{\text{(Total number of terms in document d)}}$$

- Inverse document frequency:

$$IDF(t) = \log \frac{\text{Number of documents}}{\text{containing term t}}$$

- TF-IDF score:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$



notes.arpanadhikari7.com.np

common words get low scores

Rare, meaningful words get higher scores

By: Manish Awale

Original Documents:

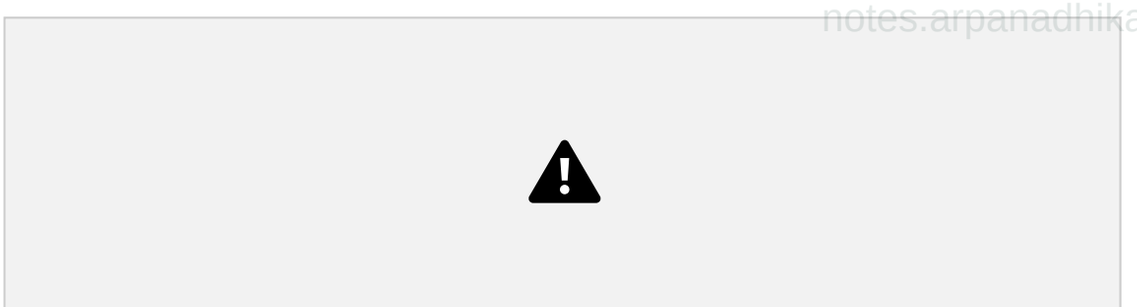
1. The cat sat on the mat
2. The dog sat on the log
3. The cat and dog are friends
4. Machine learning is fascinating
5. Deep learning uses neural networks

'The cat sat on the mat'
Total words in document: 6
Word frequencies:

TF-IDF Example

Document 1:

notes.arpanadhikari7.com.np



Embedding vector:

[0 0 0.1527 0 0 0 0 0 0 0 0 0.2682 0 0 0.1527 0.1527 0.1703 0]

Embedding Operations

- Various operations that can be performed on embeddings.

○ ***Similarity***

- Vector norm:

- L_1 (Manhattan): $\|x\|_1 = |x_1| + |x_2| + \ldots. + |x_n|$
- L2 (Euclidean): $\|x\|_2 = \sqrt{(x_1^2 + \ldots. + x_n^2)}$
- Lp (General): $\|x\|_p = (x_1^2 + \ldots. + x_n^2)^{1/p}$
- L_∞ (Maximum): $\|x\|_\infty = \max(x_1, \ldots.., x_n)$

- Cosine similarity: Cosine similarity is a similarity measure between two tokens t_1 and t_2 that considers the angle formed by their associated embeddings.

- similarity = $\cos(\theta) = \frac{t_1 \cdot t_2}{\|t_1\| \|t_2\|}$



By: Manish Awale

Available on notes.arpanadhikari7.com.np

○ *Dimension Reduction*

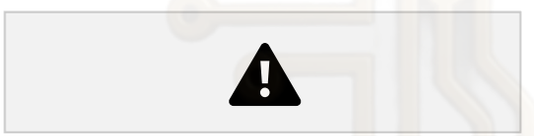
Visualization of high-dimensional embeddings into a lower dimensional space

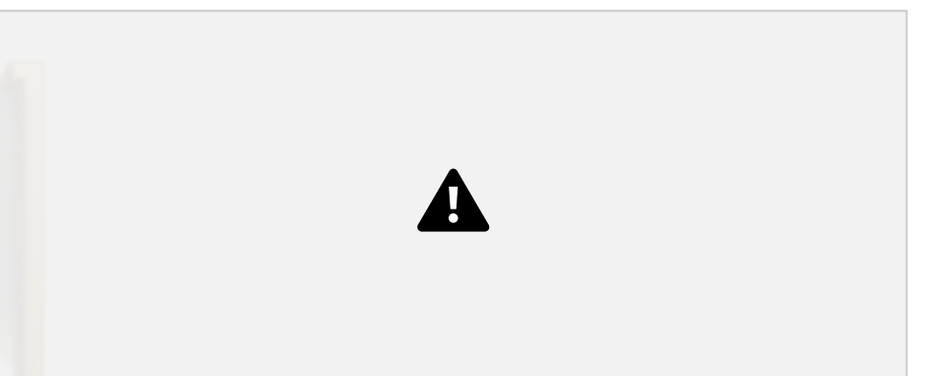
○ *t-SNE (t-distributed Stochastic Neighbor Embedding)*

similarities between data points as seen in the original high-dimensional space using a similarity score that relies on the t-distribution.

[redacted]

- Pairwise distance 
- Compute similarity scores between observations using a

Gaussian distribution 



- For each observation x_i,

compute conditional probability $p(j|i)$, for picking x_j as

neighbor of x_i 

notes.arpanadhikari7.com.np



Symmetrized probabilities 

Find a low-dimensional representation that preserves

By: Manish Awale

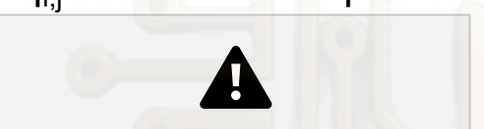
- ***t-SNE***

[redacted]

- Initialize low-dimensional observations [redacted]
  - Compute similarity scores between observations using a t-distribution with one degree of freedom



  - Compute pairwise similarities $q_{i,j}$ between each pair of observations y_i and y_j using K_d



[redacted]



notes.arpanadhikari7.com.np

Use gradient descent to decrease the loss until convergence.

Thank You

By: Manish Awale

notes.arpanadhikari7.com.np

By: Manish Awale